DC Charging System (BEVC-DC001)-II

Introduction

The rapid expansion of electric vehicles (EVs) necessitates robust and standardized charging infrastructure. This lecture delves into the intricacies of the DC charging system, specifically focusing on the BEVC-DC001 standard, and the critical role of Controller Area Network (CAN) communication in its operation. Understanding the BEVC-DC001 system, introduced by the Department of Heavy Industries, Government of India, is crucial for developing efficient and reliable EV charging solutions. It supports various battery systems and power levels, making it a versatile standard for DC fast charging.

At the heart of the BEVC-DC001 system lies CAN communication, a widely adopted protocol in the automotive industry. This lecture explores the fundamental principles of CAN, from its electrical properties and data frame structures to the sophisticated arbitration process that ensures seamless data exchange between the charger and the vehicle's Battery Management System (BMS). By dissecting how messages are prioritized, transmitted, and interpreted, we gain insight into the reliability and flexibility that CAN brings to the demanding environment of EV charging.

Core Concepts

Term	Definition	Significance
`BEVC-DC001`	A DC charging standard supporting 48V and 72V battery systems with power levels up to 10kW and 15kW respectively.	Standardizes DC charging in India, ensuring interoperability between chargers and EVs.
`CAN Communication`	A two-wire, message-based serial communication protocol used for real-time data exchange in automotive and industrial applications.	Provides a robust, flexible, and cost-effective method for charger and vehicle (BMS) to communicate critical charging parameters.
`Dominant Logic (Logic 0)`	A state on the CAN bus where the differential voltage is high	Essential for arbitration, as a node transmitting dominant logic wins bus

Term	Definition	Significance
	(e.g., 2V), overriding recessive logic.	access over those transmitting recessive logic.
`Recessive Logic (Logic 1)`	A state on the CAN bus where the differential voltage is low (e.g., 0V), easily overridden by dominant logic.	Represents the default or 'idle' state, and is used by nodes that lose arbitration.
`Message ID`	A unique identifier within a CAN message that also determines its priority; lower ID means higher priority.	Crucial for arbitration, allowing the highest priority message to gain control of the bus without data collision.
`Arbitration`	The process by which multiple nodes simultaneously attempting to transmit on the CAN bus resolve contention, with the highest priority message winning.	Ensures that critical messages (e.g., safety-related) are transmitted without delay, maintaining system integrity.
`CAN Data Frame`	The structured format in which data is transmitted over the CAN bus, including identifier, data, and error checking fields.	Defines how information is packaged and exchanged, enabling reliable and error-free communication.
`DLC (Data Length Code)`	A 4-bit field within a CAN data frame indicating the number of bytes (0-8) in the data field.	Informs receiving nodes about the size of the data payload, aiding in correct data interpretation.
`CRC (Cyclic Redundancy Check)`	A 16-bit error detection code embedded in each CAN message.	Ensures the integrity of transmitted data, allowing receivers to detect and reject corrupted messages.
`CAN Database File`	A text file containing detailed information about CAN messages and their signals,	Enables nodes to correctly interpret raw data from CAN

Term	Definition	Significance
	including scaling factors, offsets, and units.	messages into meaningful engineering units.

Detailed Analysis

1. BEVC-DC001 Charging System and Communication Architecture

The BEVC-DC001 standard defines a specific DC charging system with clear operational parameters and communication protocols.
- **Power Specifications**:
- Supports 48V battery systems with a maximum current of 200A, leading to a maximum power of 10kW.
- Supports 72V battery systems with a maximum current of 200A, leading to a maximum power of 15kW.
- **Input**: The system operates with a 3-phase AC input.
- **Communication Standards**:
- Derived from IEC 61851, specifically AIS 138 part 2, Annexure G.
- Utilizes the GB/T 27930 application layer, which defines how CAN data frames are represented.
- The physical and data link layers are based on SAE J1939 and ISO 11898 standards.
- **Connector**: Employs the GB/T 20234.3 connector with a standard 5-meter cable.
- **CAN Communication Details**:
- **Speed**: Operates at 250 kbps.
- **Identifier**: Uses CAN 2.0b identifier, which is a 29-bit identifier, allowing for 2^29 distinct messages.
- **Participants**: A dedicated communication between only two participants: the charger and the vehicle's BMS.
- **Addresses**:
- Charger address: 86 (decimal) or 56h (hexadecimal).
- Vehicle/BMS address: 244 (decimal) or F4h (hexadecimal).
- **Isolation**: This CAN communication is independent and does not interfere with other in-vehicle CAN systems like powertrain CAN or ECU CAN, ensuring dedicated and reliable charging communication.

2. Fundamentals of CAN Communication

CAN (Controller Area Network) is a robust and widely adopted serial communication protocol, particularly in the automotive industry, due to several key advantages:

- **Two-Wire System**: Consists of CAN High (CANH) and CAN Low (CANL) wires, over which data is sent differentially.
- **Differential Signaling**: The actual data is derived from the voltage difference between CANH and CANL. This method significantly enhances robustness against external electromagnetic interference (EMI).
- **Logic States**:
- **Logic 0 (Dominant)**: Detected when the differential voltage (Vdiff = CANH - CANL) is greater than or equal to 0.9V. When a node transmits dominant logic, CANH is typically 3.5V and CANL is 1.5V.
- **Logic 1 (Recessive)**: Detected when the differential voltage is less than or equal to 0.5V. When a node transmits recessive logic, both CANH and CANL are typically at 2.5V.
- **Dominance**: If any node transmits a dominant logic (0), the entire bus goes to dominant logic, overriding any recessive logic (1) transmitted by other nodes.
- **Physical Layer**:
- **Cables**: CANH and CANL are typically shielded twisted pair cables, which further reduces interference to nearby circuits and improves communication reliability.
- **Terminating Resistors**: Two 120Ω terminating resistors are placed at each end of the CAN bus. These resistors match the impedance of the network, mitigating signal integrity issues like reflections, rise time, and fall time, especially at higher speeds.
- **Open Collector Configuration**: CAN nodes operate like open collector configurations. They can actively pull the CAN bus to a dominant logic (0) but cannot force it to a recessive logic (1) if another node is transmitting dominant. The bus only becomes recessive (1) if all connected nodes are transmitting recessive logic.
- **Advantages**: Low cost (reduces wiring), centralized communication, robust against EMI, efficient, error detection (CRC), flexible (software-based modifications), and multi-master configuration.
- **Message Priority**: A fundamental aspect of CAN is that messages have priorities defined by their Message ID. A lower numerical Message ID indicates a higher priority.

3. CAN Data Frame Structures (CAN 2.0a and CAN 2.0b)

CAN communication utilizes specific data frame formats to transmit information. There are two main types:
- **CAN 2.0a (Standard Data Frame)**: Uses an 11-bit identifier.
- **SOF (Start of Frame)**: A single dominant bit (0) used to synchronize nodes and indicate the start of a new message.
- **Identifier**: An 11-bit field that establishes message priority. Lower binary value means higher priority.
- **RTR (Remote Transmission Request)**: A single bit.

- Dominant (0): Indicates a data frame (contains data).
- Recessive (1): Indicates a remote request frame (requests data from another node with a matching Message ID).
- **IDE (Identifier Extension)**: A single bit.
- Dominant (0): Indicates a standard CAN 2.0a data frame (no extension).
- **r0 (Reserved Bit)**: Reserved for future use.
- **DLC (Data Length Code)**: A 4-bit field specifying the number of data bytes (0 to 8) in the frame.
- **Data Field**: Contains 0 to 8 bytes (0 to 64 bits) of application data. This can be configured in various ways (e.g., 64 signals of 1 bit each, 8 signals of 8 bits each, or 1 signal of 64 bits).
- **CRC (Cyclic Redundancy Check)**: A 16-bit field for error detection.
- **ACK (Acknowledgement)**: A 2-bit field (1 ACK bit, 1 delimiter). The sender transmits a recessive ACK bit, which is overwritten by a dominant bit by any receiver that successfully receives an error-free message (verified by CRC).
- **EOF (End of Frame)**: A 7-bit field marking the end of the CAN frame.
- **IFS (Inter Frame Space)**: A 7-bit field providing time for the receiver to move the received message into its buffer.

    - **CAN 2.0b (Extended Data Frame)**: Uses a 29-bit identifier, allowing for a much larger number of unique messages (2^29).
    - **SOF**: Same as CAN 2.0a.
    - **11-bit Identifier**: Same as CAN 2.0a.
    - **SRR (Substitute Remote Request)**: A recessive bit that indicates the CAN message is in extended format, allowing nodes to recognize extended frames without needing the full 29-bit identifier.
    - **IDE (Identifier Extension)**: A recessive bit indicating that more identifier bits follow.
    - **18-bit Identifier**: An additional 18-bit field that, combined with the initial 11-bit identifier, forms the full 29-bit identifier. This 29-bit value determines the message priority.
    - **r1 (Reserved Bit)**: Reserved for future expansion.
    - **RTR, DLC, Data, CRC, ACK, EOF, IFS**: These fields function similarly to CAN 2.0a.

4. Arbitration: Resolving Bus Contention

Arbitration is the mechanism by which multiple nodes simultaneously attempting to transmit on the CAN bus resolve contention, ensuring that only one message is successfully transmitted at a time.

- **Process**:
- When multiple nodes start transmitting, they all send their Message ID bit by bit, starting with the most significant bit.
- If a node transmits a recessive bit (1) but detects a dominant bit (0) on the bus, it immediately stops transmitting and becomes a receiver. This is because a dominant bit on the bus indicates that another node is transmitting a message with a higher priority (lower Message ID, meaning more leading zeros).
- The node that continues to transmit dominant bits for the longest duration (i.e., has the lowest Message ID) wins the arbitration and gains control of the bus.
- **Example**: If Node 1 (ID 000...), Node 2 (ID 001...), and Node 3 (ID 010...) all try to transmit:
- All send the first bit (0): Bus is 0 (dominant).
- All send the second bit (0): Bus is 0 (dominant).
- Node 1 sends 0, Node 2 sends 1, Node 3 sends 0: Bus is 0 (dominant). Node 2 detects a dominant 0 while sending a recessive 1, so Node 2 loses arbitration and becomes a receiver.
- Node 1 sends 0, Node 3 sends 1: Bus is 0 (dominant). Node 3 detects a dominant 0 while sending a recessive 1, so Node 3 loses arbitration and becomes a receiver.
- Node 1 wins arbitration and continues transmitting its message.
- **Outcome**: The message with the lowest Message ID (highest priority) always wins arbitration, ensuring critical data is prioritized. Once a node wins, it transmits its entire message, and then other nodes can attempt arbitration again.

5. The Role of the CAN Database File

The CAN database file (often a text file) is indispensable for interpreting the raw data transmitted over the CAN bus into meaningful engineering units.
- **Purpose**: It acts as a dictionary for all CAN messages and their embedded signals, allowing any connected node to understand the data it receives.
- **Contents**: For each CAN message, the database stores:
- **Message Name, Message ID, and DLC**: Basic identification of the message.
- **Signal Name**: Descriptive name for each piece of data within the message.
- **Location and Size**: Specifies the start bit and length (in bits) of each signal within the data field.
- **Signal Type**: Defines the data type (e.g., character, integer).
- **Byte Order**: Indicates whether the data is little-endian or big-endian.
- **Scaling Factor and Offset**: Parameters used to convert raw received data into actual engineering units.
- `Actual Data = (Raw Data * Scaling Factor) + Offset`
- **Unit String**: The engineering unit of the signal (e.g., Volts, Amps, Celsius).
- **Range**: Minimum and maximum expected values for the signal.

- **Functionality**: When a node receives a CAN message, it extracts the Message ID. It then consults its CAN database to check if that Message ID is relevant. If it is, the node uses the stored signal definitions (location, size, scaling, offset, unit) to correctly parse the raw data bits into meaningful information. If the Message ID is not in its database, the node ignores the message.
- **Application in DC001**: Both the EV (BMS) and the EVSE (charger) in a DC001 system possess a CAN database. This ensures that they can accurately interpret each other's messages at different stages of the charging process, facilitating proper control and monitoring.

Key Takeaways

- [ ] BEVC-DC001 is a DC charging standard supporting 48V/10kW and 72V/15kW systems, using GB/T 27930 for communication.
- [ ] CAN communication in DC001 is dedicated between the charger (address 86) and BMS (address 244), independent of other vehicle CAN systems.
- [ ] CAN is a two-wire differential signaling system (CANH/CANL) robust against EMI, using 120Ω terminating resistors.
- [ ] Logic 0 (dominant) overrides Logic 1 (recessive) on the CAN bus, a principle fundamental to arbitration.
- [ ] CAN 2.0a uses an 11-bit identifier, while CAN 2.0b uses a 29-bit identifier, with lower Message IDs indicating higher priority.
- [ ] Arbitration ensures that the message with the highest priority (lowest Message ID) gains control of the bus without data collisions.
- [ ] CAN data frames include fields like SOF, Identifier, RTR, IDE, DLC, Data, CRC, ACK, EOF, and IFS for reliable transmission.
- [ ] The CAN database file is crucial for nodes to interpret raw CAN message data into meaningful engineering units using scaling factors and offsets.
- [ ] Each CAN message can carry up to 8 bytes (64 bits) of data, which can be configured as multiple signals of varying lengths.

Conclusion

The detailed exploration of the BEVC-DC001 charging system and its underlying CAN communication protocol highlights the sophisticated engineering required for modern EV infrastructure. The standardization provided by BEVC-DC001 ensures compatibility and efficiency, while CAN communication delivers the necessary reliability, flexibility, and real-time data exchange capabilities. From the electrical properties that make CAN robust against interference to the intricate arbitration

process that prioritizes critical messages, every aspect is designed to ensure safe and effective charging.

The use of CAN database files further empowers intelligent communication, allowing the charger and vehicle to precisely understand and react to each other's status and requests. This foundational understanding of CAN's operational principles and data structures is indispensable for anyone involved in the design, implementation, or maintenance of electric vehicle charging systems, paving the way for a more connected and sustainable transportation future.